

Do Script Python ao Kubernetes na AWS

Um Guia Didático e Descomplicado de DevOps e Cloud

Estudo de Caso Prático: API de Analytics do Coworking Space

Autor: Eugênio | **Público-Alvo:** Analistas de Dados, Desenvolvedores e Iniciantes em Nuvem | October 2, 2026

Prefácio: O que este guia resolve?

Se você vem da área de **Análise de Dados**, seu dia a dia geralmente envolve consultas SQL, manipulação de DataFrames com Pandas, criação de modelos e geração de relatórios. No entanto, quando essa análise precisa ser disponibilizada para centenas de pessoas como uma API rápida e estável, entramos no mundo de **DevOps e Engenharia de Nuvem**.

Este documento conta, passo a passo e sem jargões desnecessários, como pegamos uma aplicação em Python que consulta um banco PostgreSQL e a transformamos em uma solução profissional, resiliente e 100% automatizada na **Amazon Web Services (AWS)** utilizando **Docker**, **Kubernetes (EKS)** e **Pipelines de CI/CD**.

1 O Mapa Mental dos Conceitos (Analogias do Mundo Real)

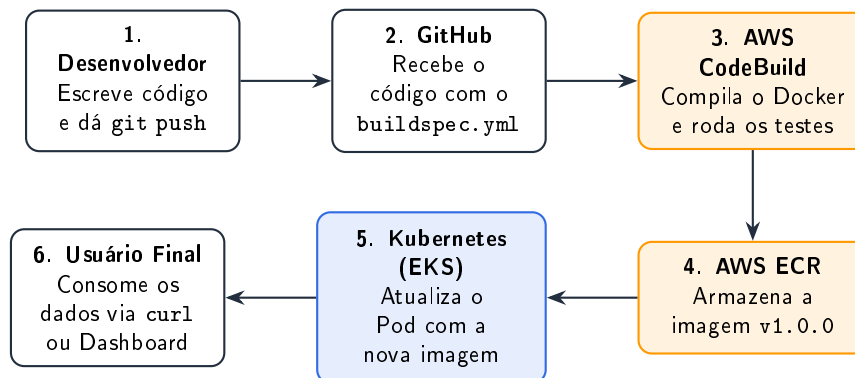
Antes de mergulhar nos comandos, vamos traduzir os termos técnicos de DevOps para conceitos intuitivos:

- **Docker (O Contêiner):** Imagine que você criou um script Python no Jupyter Notebook com bibliotecas específicas e precisa mandar para um colega. Se o colega tiver outra versão do Python ou do Windows, tudo quebra. O Docker é uma “marmita lacrada” (contêiner) que carrega o seu código, a versão exata do Python, as bibliotecas do `requirements.txt` e até o sistema operacional mínimo. Onde essa marmita for aberta (no seu notebook, no servidor do chefe ou na nuvem), o script roda exatamente da mesma forma.
- **AWS ECR (O Armazém de Imagens):** É como um repositório seguro no Google Drive ou GitHub, mas específico para guardar essas “marmitas lacradas” (imagens Docker) prontas para serem usadas.
- **AWS CodeBuild (A Linha de Montagem / CI):** Em vez de você ficar compilando a imagem no seu computador toda vez que mudar uma linha de código, o CodeBuild é um computador descartável na nuvem que baixa seu código do GitHub, prepara a imagem Docker e a coloca no armazém (ECR) automaticamente.
- **Kubernetes (O Maestro da Orquestra):** Se sua marmita (contêiner) travar de madrugada ou se o servidor queimar, quem acorda para reiniciar? O Kubernetes. Ele monitora a saúde das aplicações, divide o trabalho entre computadores e garante que, se um processo morrer, outro seja criado em questão de segundos sem ninguém perceber.
- **AWS EKS (A Infraestrutura do Maestro):** É o serviço da AWS que monta toda a sala de máquinas do Kubernetes para você com segurança e alta disponibilidade.
- **Helm (O Instalador de Pacotes do Kubernetes):** Pense no Helm como o `pip` do Python ou o `apt` do Linux, só que para infraestrutura. Em vez de escrever 300 linhas de configuração para subir um banco PostgreSQL, você roda um único comando do Helm e o banco sobe completo.

2 Fluxogramas Didáticos do Sistema

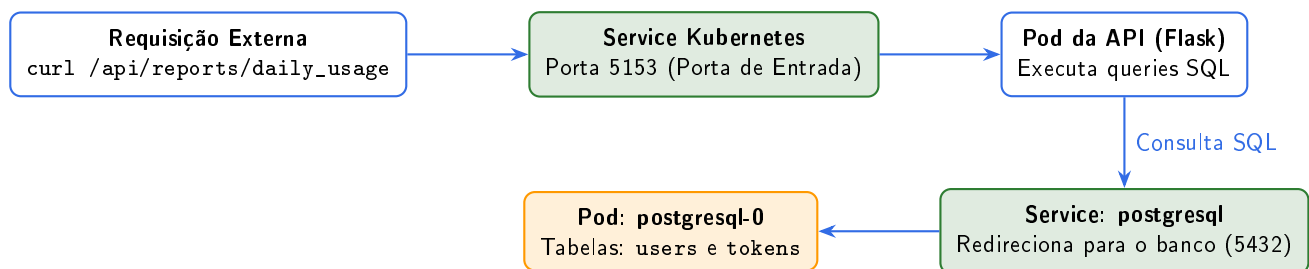
2.1 Fluxo 1: A Linha de Montagem da Aplicação (Pipeline CI/CD)

O diagrama abaixo explica o que acontece nos bastidores quando uma alteração no código é feita:



2.2 Fluxo 2: Como os Dados Chegam ao Analista (Arquitetura Interna)

Como o tráfego navega de dentro do cluster até o banco de dados PostgreSQL:



3 A Crônica Detalhada das Etapas (O Que Fizemos e Por Quê)

A seguir, veja a narrativa real e sem cortes de cada passo percorrido, com as motivações por trás de cada comando:

3.1 Etapa 1: O Ponto de Partida e o Mistério da AWS sem Chaves

O que aconteceu: Ao tentar rodar `aws sts get-caller-identity` no terminal local, recebemos o erro informando que o comando não existia. Depois de instalá-lo, surgiu o erro *“Unable to locate credentials”*.

A explicação didática: O utilitário **AWS CLI** é apenas o “telefone” que você usa para falar com os servidores da Amazon. Mas antes de a Amazon atender suas ordens, ela precisa saber quem está ligando. As variáveis de ambiente (`AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY` e `AWS_SESSION_TOKEN`) são como o seu crachá e senha temporária de acesso ao prédio da AWS. Uma vez exportadas no terminal, a AWS nos reconheceu imediatamente.

3.2 Etapa 2: Erguendo a Cidade do Kubernetes com eksctl

O que aconteceu: Executamos o comando:

```
eksctl create cluster --name my-cluster --region us-east-1 --nodegroup-name my-nodes  
--node-type t3.small --nodes 1
```

O comando levou cerca de 18 minutos rodando silenciosamente no terminal.

A explicação didática: Uma dúvida natural foi: “*esse comando fica rodando para sempre?*”. Não! Ele demorou porque o `eksctl` não instalou um simples programa; ele desenhou e ergueu uma infraestrutura inteira do zero na nuvem da AWS via CloudFormation:

1. Criou uma **VPC** (uma rede privada isolada no datacenter da AWS em Virgínia do Norte).
2. Criou sub-redes públicas e privadas com tabelas de roteamento e regras de firewall.
3. Provisionou o **Control Plane** do Kubernetes (os servidores mestres que tomam decisões).
4. Criou uma máquina virtual EC2 (`t3.small`) como nó operário (*worker node*).

Ao final, o `eksctl` salvou o arquivo `~/.kube/config`, que funciona como a chave mestra para que o comando `kubectl` possa controlar o cluster do nosso terminal.

3.3 Etapa 3: Criando e Populando o Banco de Dados com Helm

O que aconteceu: Subimos um PostgreSQL com o comando `helm install postgresql bitnami/postgresql` e em seguida populamos as tabelas com os scripts `1_create_tables.sql`, `2_seed_users.sql` e `3_seed_tokens.sql`.

A explicação didática: Para um analista, banco de dados é a alma do projeto. Em vez de criarmos manualmente máquinas para o banco, o **Helm** aplicou uma receita pronta da Bitnami. O banco gerou uma senha aleatória que ficou guardada em um objeto seguro do Kubernetes chamado **Secret**.

O obstáculo do caminho: Quando tentamos conectar na porta 5432 via *port-forward*, a senha falhava porque a máquina local já tinha outro PostgreSQL instalado ocupando a porta 5432. Resolvemos isso de forma elegante: injetamos os arquivos SQL diretamente para dentro do contêiner do banco usando o comando `kubectl exec`, sem sofrer interferência da rede local.

3.4 Etapa 4: Empacotando o Python no Docker e o Bug Oculto do Flask

O que aconteceu: Criamos o `Dockerfile` baseado em `python:3.10-slim` e o enviamos para o cluster. Mas ao analisar os logs do pod com `kubectl logs`, nos deparamos com:

```
ImportError: cannot import name 'url_quote' from 'werkzeug.urls'
```

A explicação didática: Este é um problema clássico de projetos de dados: o arquivo `requirements.txt` original dizia apenas `flask==2.2.2`. Como a biblioteca auxiliar **Werkzeug** não tinha versão travada, o pip baixou a versão mais nova (**Werkzeug 3.0**), que apagou uma função antiga que o Flask esperava. Corrigimos adicionando `Werkzeug==2.2.2`, garantindo reprodutibilidade matemática das dependências.

3.5 Etapa 5: Automatizando a Nuvem com GitHub e AWS CodeBuild

O que aconteceu: Configuramos o arquivo `buildspec.yml` e conectamos nosso repositório GitHub ao serviço **AWS CodeBuild**.

A explicação didática: O CodeBuild atua como um assistente de fábrica. Toda vez que iniciamos a compilação, ele aciona uma máquina virtual na AWS, lê as instruções do `buildspec.yml`, constrói a imagem Docker, adiciona as etiquetas de versão (`1.0.0` e `latest`) e publica a imagem no **AWS ECR**. A partir desse momento, qualquer servidor do nosso cluster tem acesso direto à imagem oficial da aplicação.

3.6 Etapa 6: O Mistério do Pod que Não Ficava Pronto (Readiness Probe)

O que aconteceu: Após o build, o comando `kubectl rollout status` ficou travado com a mensagem: “*1 old replicas are pending termination...*”, e o pod novo aparecia como `0/1 Running`.

A explicação didática: No Kubernetes, um pod não recebe clientes só porque começou a rodar. O Kubernetes possui “sensores médicos” chamados **Probes**:

- **Liveness Probe:** Pergunta se a aplicação está viva. Se responder não, o Kubernetes mata e recria o pod.

- **Readiness Probe:** Pergunta: “*Você já está pronto para receber tráfego de clientes?*”.

Nosso manifesto verificava a rota `/readiness_check`, mas dentro do código Python ela tentava fazer `db.session.query(Token).count()` sem a classe `Token` existir, devolvendo erro 500. Como o pod novo nunca dizia “estou pronto”, o Kubernetes recusava-se a desligar o pod antigo para evitar que os usuários ficassem sem sistema! Corrigimos o sensor para apontar para a rota funcional `/health_check`, e o pod imediatamente assumiu o status **1/1 Ready**.

3.7 Etapa 7: O Gran Finale – Relatórios de Negócio Respondendo 100%

O que aconteceu: Abrimos o túnel na porta 5153 e executamos as consultas HTTP. Encontramos um último detalhe na rota `/api/reports/daily_usage` (onde estava escrito `jsonify(get_daily_visits)` sem os parênteses da função, gerando erro 500). Corrigido para `jsonify(get_daily_visits())`, as duas rotas responderam instantaneamente:

```
{ "2023-02-07": 40, "2023-02-08": 202, "2023-02-09": 179, "2023-02-10": 158, ... }
```

O microserviço consultou os 500 registros de check-ins no PostgreSQL e entregou a agregação em JSON puro para consumo dos analistas!

4 Decisões de Engenharia e Boas Práticas (Standout Topics)

1. Por que definir Requests e Limits de CPU e Memória?

Em análise de dados, um script mal dimensionado pode tentar carregar um dataset de 10 GB na memória e travar o computador inteiro. No Kubernetes evitamos isso com:

- **Requests** (`cpu: 100m, memory: 64Mi`): É o “bilhete mínimo” que o pod precisa para entrar no ônibus (servidor). Garante que a aplicação sempre tenha recursos reservados.
- **Limits** (`cpu: 250m, memory: 256Mi`): É o “teto de gastos”. Se o contêiner tentar usar mais de 256 MB de RAM, o Kubernetes o freia antes que ele prejudique os outros pods da máquina.

2. Escolha da Instância EC2: Por que a família Graviton (t4g)?

Recomendamos instâncias do tipo `t4g.small`. Elas utilizam os processadores **AWS Graviton2**, desenhados pela própria Amazon com arquitetura ARM (semelhante aos chips de celulares modernos e Macs Apple Silicon). Elas entregam cerca de **40% mais velocidade** consumindo menos energia e com um **custo 20% menor** que instâncias convencionais Intel/AMD.

3. Como Economizar Dinheiro Real na AWS?

1. **Instâncias Spot:** A AWS leiloa servidores ociosos com descontos de até 90%. Para ambientes de homologação ou tarefas batch de dados, usar instâncias Spot com a ferramenta *Karpenter* reduz drasticamente a fatura no fim do mês.
2. **Autoescalabilidade Inteligente (HPA):** Durante o horário comercial a empresa tem muitos analistas gerando relatórios (o Kubernetes sobe 5 réplicas do pod). À noite, ele reduz para apenas 1 pod, economizando infraestrutura.

5 Conclusão da Jornada

O que começou como um punhado de scripts locais transformou-se em uma arquitetura moderna de microsserviços na nuvem. Compreender esses passos permite ao profissional de dados transitar com confiança entre a extração de *insights* e a engenharia robusta que sustenta os produtos em escala mundial.